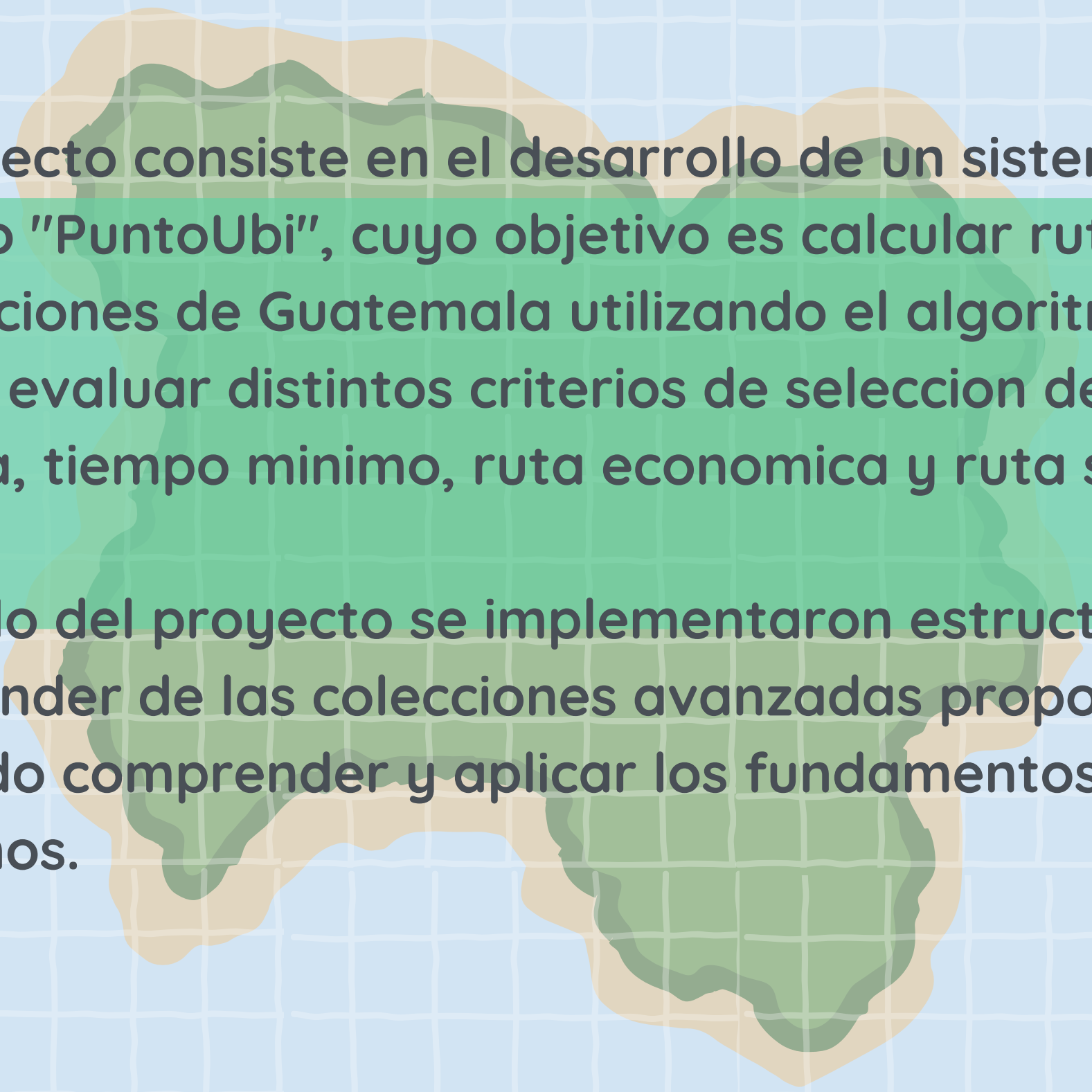


INFORME TECNICO

PUNTO VBI

A stylized map of Guatemala is centered in the background. It features a green landmass with darker green mountain ranges and a tan-colored border representing the surrounding sea. The map is set against a light blue background with a white grid pattern. Decorative blue wavy lines are placed in the top-left and bottom-left corners.

El presente proyecto consiste en el desarrollo de un sistema de navegacion GPS denominado "PuntoUbi", cuyo objetivo es calcular rutas optimas entre diferentes ubicaciones de Guatemala utilizando el algoritmo de Dijkstra. El sistema permite evaluar distintos criterios de seleccion de rutas, como distancia minima, tiempo minimo, ruta economica y ruta segura.

Para el desarrollo del proyecto se implementaron estructuras de datos propias sin depender de las colecciones avanzadas proporcionadas por Java, permitiendo comprender y aplicar los fundamentos de estructuras de datos y algoritmos.

2. ESTRUCTURAS DE DATOS UTILIZADAS

2.1 Lista Dinamica

Se implemento una estructura denominada Lista basada en arreglos dinamicos.

Funciones principales:

- Agregar elementos.
- Obtener elementos por indice.
- Eliminar elementos.
- Buscar elementos.
- Expandir automaticamente la capacidad cuando se llena.

Aplicaciones dentro del proyecto:

- Almacenar ubicaciones.
- Almacenar conexiones entre nodos.
- Guardar pasos del algoritmo de Dijkstra.
- Construir rutas resultantes.

Complejidad:

- Agregar: $O(1)$ amortizado.
- Buscar: $O(n)$.
- Obtener por indice: $O(1)$.
- Eliminar: $O(n)$.

2. ESTRUCTURAS DE DATOS UTILIZADAS

2.2 Cola (FIFO)

La estructura Cola fue implementada mediante nodos enlazados.

Características:

- El primer elemento en entrar es el primero en salir.
- Permite operaciones de encolar y desencolar.

Complejidad:

- Encolar: $O(1)$.
- Desencolar: $O(1)$.
- Ver frente: $O(1)$.

2. ESTRUCTURAS DE DATOS UTILIZADAS

2.3 Pila (LIFO)

La estructura Pila fue implementada utilizando nodos enlazados.

Características:

- El ultimo elemento ingresado es el primero en salir.
- Permite apilar y desapilar elementos.

Uso en el proyecto:

- Reconstruccion de la ruta encontrada por Dijkstra desde el destino hasta el origen.

Complejidad:

- Apilar: $O(1)$.
- Desapilar: $O(1)$.
- Ver tope: $O(1)$.

2. ESTRUCTURAS DE DATOS UTILIZADAS

2.4 Cola de Prioridad

Se implemento una ColaPrioridad basada en un Heap Minimo.

Funcion:

- Mantener ordenados los nodos segun la menor distancia acumulada.
- Permitir seleccionar rapidamente el siguiente nodo a procesar.

Uso en el proyecto:

- Es la estructura fundamental del algoritmo de Dijkstra.

Complejidad:

- Insertar: $O(\log n)$.
- Extraer minimo: $O(\log n)$.



2. ESTRUCTURAS DE DATOS UTILIZADAS

2.5 Mapa (Tabla Hash)

La estructura Mapa<K,V> fue implementada mediante una tabla hash con encadenamiento.

Uso dentro del proyecto:

- Almacenar distancias calculadas.
- Registrar predecesores.
- Asociar tramos seleccionados.

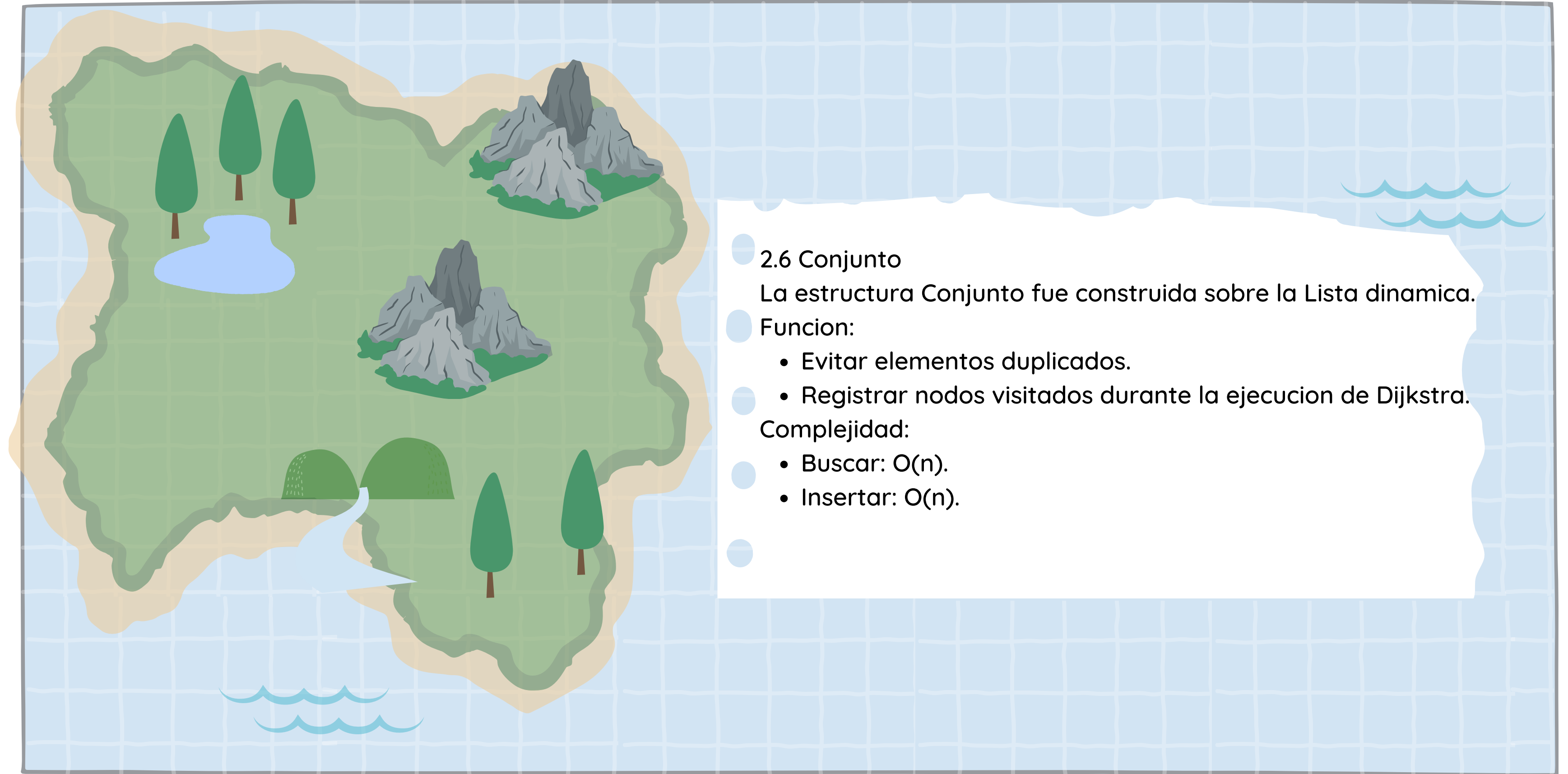
Complejidad promedio:

- Insertar: $O(1)$.
- Buscar: $O(1)$.
- Actualizar: $O(1)$.

Peor caso:

- $O(n)$.

2. ESTRUCTURAS DE DATOS UTILIZADAS



2.6 Conjunto

La estructura Conjunto fue construida sobre la Lista dinamica.

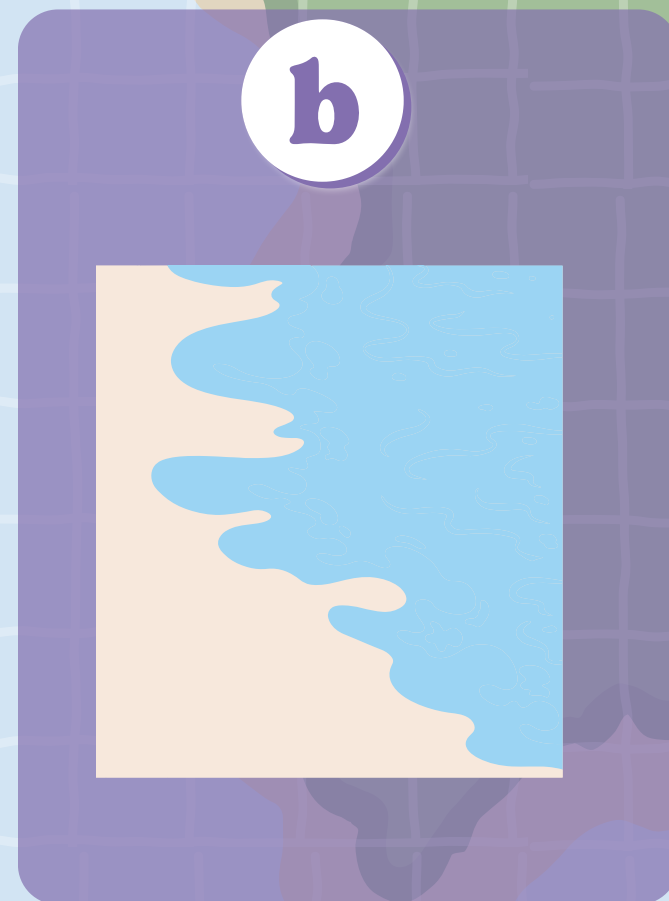
Funcion:

- Evitar elementos duplicados.
- Registrar nodos visitados durante la ejecucion de Dijkstra.

Complejidad:

- Buscar: $O(n)$.
- Insertar: $O(n)$.

3. REPRESENTACION DEL GRAFO



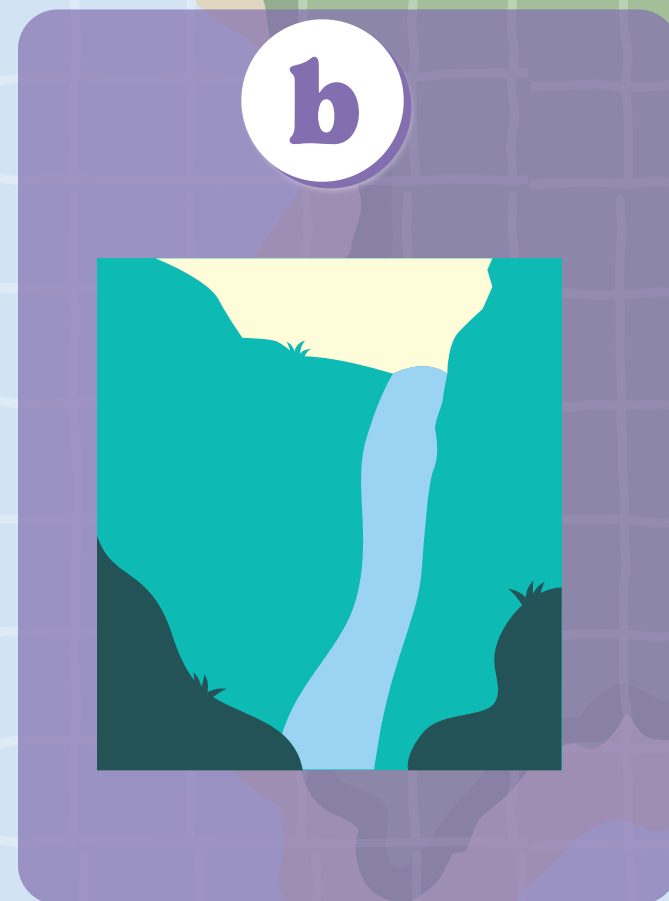
La red vial fue modelada mediante un grafo no dirigido utilizando listas de adyacencia.

Nodos

Los nodos son representados por la clase Ubicacion, la cual almacena:

- Codigo unico.
- Nombre comercial.
- Latitud.
- Longitud.
- Altitud.

3. REPRESENTACION DEL GRAFO



Aristas

Las conexiones son representadas por la clase TramoConexion.

Cada arista almacena:

- Distancia en kilometros.
- Velocidad promedio.
- Tipo de via.
- Presencia de peaje.
- Clasificacion de ruta.

Ventajas de las listas de adyacencia

- Menor consumo de memoria.
- Mayor eficiencia para grafos dispersos.
- Facil administracion de conexiones.

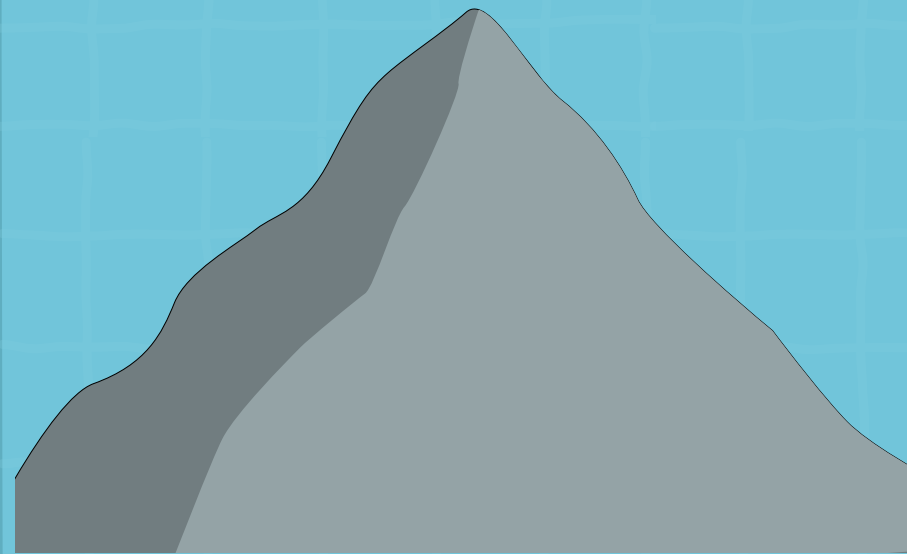
Complejidad:

- Espacio: $O(V + E)$

Donde:

- V = cantidad de vertices.
- E = cantidad de aristas.

4. ALGORITMO PRINCIPAL



Algoritmo de Dijkstra

El sistema utiliza el algoritmo de Dijkstra para encontrar la mejor ruta entre dos ubicaciones.

Proceso general:

1. Inicializar todas las distancias como infinito.
2. Asignar distancia cero al nodo origen.
3. Insertar el origen en la cola de prioridad.
4. Extraer continuamente el nodo con menor costo.
5. Relajar las aristas vecinas.
6. Actualizar distancias y predecesores.
7. Reconstruir la ruta final utilizando una pila.

Ventajas:

- Garantiza encontrar la ruta optima cuando los pesos son positivos.
- Es ampliamente utilizado en sistemas GPS reales.

Complejidad:

Utilizando cola de prioridad:

$O((V + E) \log V)$

donde:

- V representa la cantidad de nodos.
- E representa la cantidad de conexiones.

5. DECISIONES DE DISEÑO



Durante el desarrollo del proyecto se tomaron las siguientes decisiones:

Implementación de estructuras propias

Se decidió implementar manualmente las estructuras de datos para comprender su funcionamiento interno y cumplir con los objetivos académicos del proyecto.

Uso de listas de adyacencia

Se eligió esta representación porque las redes viales poseen muchas menos conexiones que las que tendría una matriz completa, reduciendo considerablemente el consumo de memoria.

Uso de coordenadas geográficas reales

La distancia entre ubicaciones se calcula mediante la fórmula de Haversine, permitiendo obtener distancias aproximadas sobre la superficie terrestre.

Incorporación de tráfico y condiciones especiales

El sistema no solamente considera la distancia física, sino también:

- Hora del día.
- Tráfico vehicular.
- Lluvia.
- Accidentes.
- Construcciones.
- Eventos especiales.

Esto genera resultados más cercanos a un sistema GPS real.

VISUALIZACIÓN

Nombre de la ciudad (Nodo)

Latitud y Longitud de la ciudad

Inicio del recorrido (Nombre de la ciudad)

Distancia a tomar del recorrido, y el tipo de movilización

AGREGAR CIUDAD

NOMBRE

LATITUD

LONGITUD

ALTITUD (M)

+ Agregar Ciudad

CONECTAR CIUDADES

ORIGEN

DESTINO

DISTANCIA (KM) - OPCIONAL

↔ Conectar

Barra de desplazamiento

Altitud de la ciudad

Aquí es para crear el recorrido

Destino del recorrido

Botón para conectar ciudades

VISUALIZACIÓN

Inicio del recorrido (Nombre de la ciudad)

Botón para iniciar el recorrido

Resultado del recorrido o conexiones

Botón para iniciar el recorrido del viaje

CALCULAR RUTA

DESDE

— seleccionar —

HASTA

— seleccionar —

► Dijkstra

Existe camino?

RESULTADO

Selecciona origen y destino para calcular.

SIMULACION DE VIAJE

► Simular paso a paso

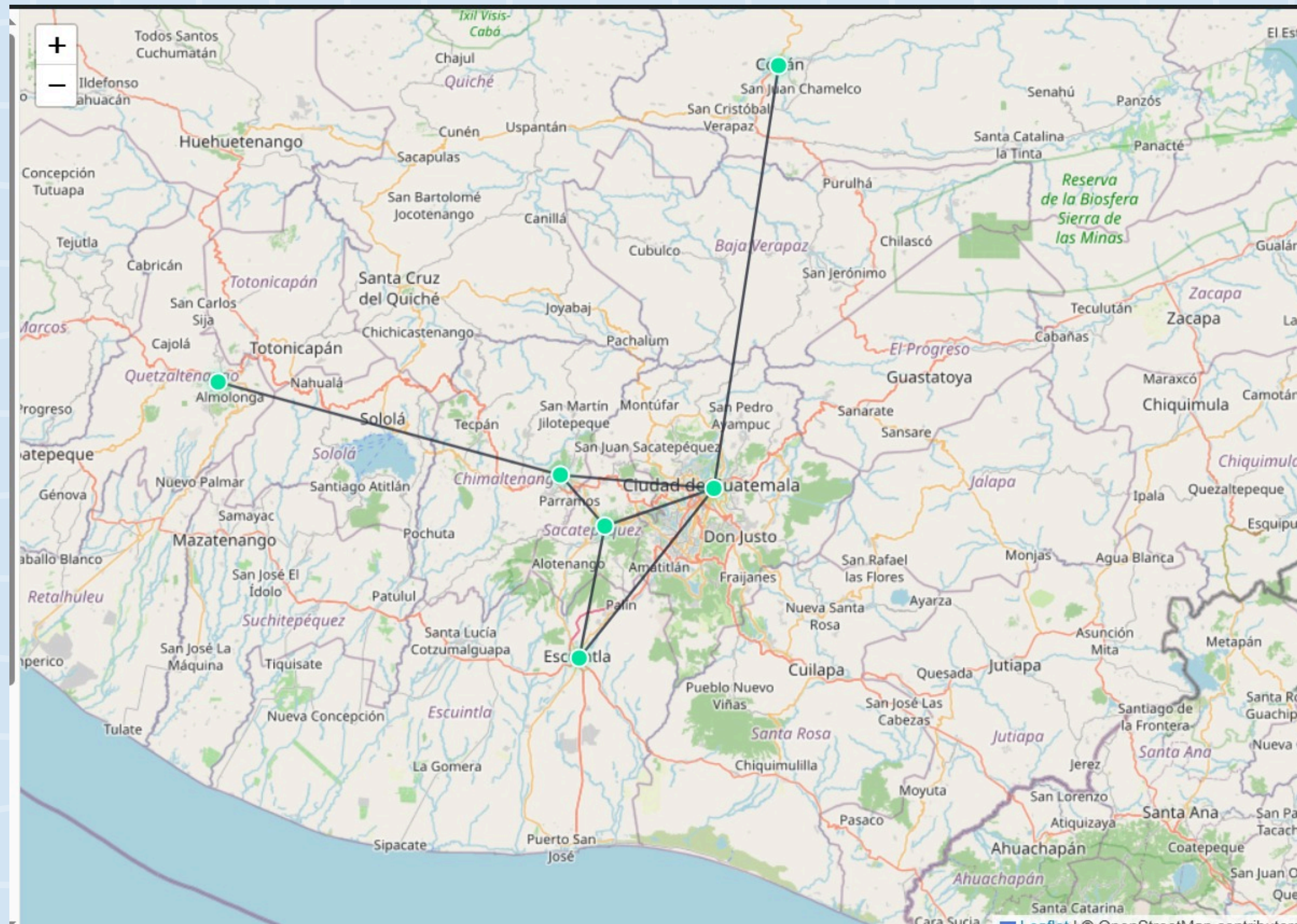
Destino del recorrido

Botón para revisar si hay camino ya existente

Aquí es para iniciar el recorrido

VISUALIZACIÓN

Mapa con el recorrido



VISUALIZACIÓN

Ciudades con sus datos, altitud, latitud, longitud.

CIUDADES (14)

- Antigua Guatemala (Centro)
14.5573, -90.7332 · 1530m
- Ciudad Vieja
14.5256, -90.7694 · 1550m
- Jocotenango
14.5850, -90.7430 · 1490m
- San Miguel Dueñas
14.5217, -90.7967 · 1580m
- Santiago Sacatepéquez
14.6356, -90.6786 · 1620m
- Pastores
14.5950, -90.7550 · 1570m
- San Juan del Obispo
14.5450, -90.7100 · 1680m
- Santa María de Jesús
14.5150, -90.7200 · 2100m
- Zona 1 - Centro Histórico
14.6407, -90.5133 · 1490m
- Zona 10 - Zona Viva
14.6030, -90.5080 · 1520m
- Cayalá
14.6110, -90.4860 · 1540m
- Aeropuerto La Aurora
14.5833, -90.5275 · 1509m
- Tikal Futura, Zona 7
14.6210, -90.5200 · 1500m
- Obrajes, Zona 10
14.5980, -90.5150 · 1530m

CONCLUSIÓN

EL PROYECTO PUNTOUBI PERMITIO APLICAR CONCEPTOS FUNDAMENTALES DE ESTRUCTURAS DE DATOS, GRAFOS Y ALGORITMOS DE BUSQUEDA DE RUTAS. LA IMPLEMENTACION DE LISTAS, PILAS, COLAS, COLAS DE PRIORIDAD, MAPAS Y CONJUNTOS HIZO POSIBLE CONSTRUIR UNA SOLUCION EFICIENTE PARA EL PROBLEMA DE NAVEGACION.

LA UTILIZACION DEL ALGORITMO DE DIJKSTRA JUNTO CON UNA REPRESENTACION MEDIANTE LISTAS DE ADYACENCIA PERMITIO OBTENER RUTAS OPTIMAS CON UN COSTO COMPUTACIONAL ADECUADO, DEMOSTRANDO LA IMPORTANCIA DE SELECCIONAR CORRECTAMENTE LAS ESTRUCTURAS DE DATOS PARA RESOLVER PROBLEMAS REALES DE MANERA EFICIENTE.